SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY

COURSE CODE: CSA5101

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: **Total Marks: 50**

Annexure A

Code of Conduct:

*I **Bharath M (192324302**) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1. Abstract

A key management system (KMS) is responsible for creating, protecting, distributing, using, rotating, revoking, backing up, and eventually destroying cryptographic keys. Encryption algorithms can be mathematically strong, but the security of the complete system can still fail when keys are stored in plaintext, protected with weak or incorrectly configured encryption, exposed through source code or logs, distributed over insecure channels, or retained longer than necessary. The assessment question describes a Python-based key management system that fails to securely store and retrieve cryptographic keys. The task is therefore not limited to correcting a programming error; it requires identifying design weaknesses and replacing them with a controlled lifecycle and defence-in-depth approach.

This report analyses the main failure modes in key storage and retrieval, including hard-coded secrets, predictable key generation, unsafe file permissions, weak password-based encryption, missing integrity checks, inadequate access control, insecure key distribution, and incomplete rotation or revocation. A systematic debugging workflow is proposed: reproduce the failure, trace key creation and retrieval, inspect configuration and permissions, define security invariants, implement the correction, and validate it using positive, negative, and lifecycle tests.

The proposed design uses authenticated encryption for stored key material, separates the encryption key from the encrypted key store, applies least-privilege access control, records auditable metadata without logging secret values, and defines states for key creation, activation, rotation, deactivation, revocation, archival, and destruction. An illustrative Python implementation uses the Fernet interface from the maintained `cryptography` package to demonstrate authenticated encryption and safe retrieval. The example is educational: production systems should prefer a cloud key-management service, hardware security module (HSM), or operating-system secret store for protecting the master key rather than placing it beside the encrypted data.

The report also explains performance and operational optimisations, proposes a test plan and evaluation metrics, and analyses how poor key management can compromise confidentiality, integrity, authentication, and availability across an entire cryptographic system. The central conclusion is that secure key management depends on the complete lifecycle, separation of duties, reliable recovery, and continuous verification—not merely on choosing a strong cipher.

2. Problem Statement and Objectives

Problem statement: A Python-based key management system cannot securely store and retrieve cryptographic keys. The likely causes include plaintext storage, improper encryption, weak access control, unsafe key distribution, and missing lifecycle controls. These failures can expose the keys that protect application data and may invalidate the security of the wider system.

2.1 Objectives

- Identify technical and design flaws in key storage, encryption, retrieval, and access control.
- Debug the system using a repeatable process and establish clear security requirements.
- Implement authenticated encryption for stored key material while keeping the protecting key separate.
- Define secure key generation, distribution, rotation, revocation, backup, recovery, and destruction procedures.
- Optimise retrieval and operational performance without weakening cryptographic controls.
- Test normal operation, invalid inputs, unauthorised access, corruption, rotation, and failure recovery.

3. Background: Role of a Key Management System

Cryptography uses keys to control the operation of algorithms. A symmetric key may encrypt and decrypt data; a public/private key pair may support encryption, key agreement, or digital signatures. A key management system governs the full process around these keys. It must ensure that only authorised identities can use a key for an approved purpose, that keys are not altered or substituted unnoticed, and that old or compromised keys can be disabled.

3.1 Why key management is as important as the algorithm

Suppose an application encrypts customer records with a modern authenticated-encryption algorithm. If the encryption key is stored in the same folder as the encrypted database and both files are readable by every local user, an attacker who obtains the folder may acquire both the ciphertext and the key. The algorithm may be correct, but the system has lost confidentiality. Similarly, if a signing private key is exposed, an attacker may generate signatures that appear to come from the legitimate organisation. This illustrates the principle that security depends on the implementation, key custody, permissions, and operational processes as well as the algorithm.

3.2 Key categories and separation

Key category	Purpose	Important control
Data-encryption key (DEK)	Encrypts application data or a stored secret.	Restrict use to the intended data and service.
Key-encryption key (KEK) / master key	Protects or wraps other keys.	Store separately in a KMS, HSM, or secret manager.
Public/private key pair	Supports key agreement, encryption, or digital signatures depending on the algorithm.	Protect private key; distribute public key with authenticity checks.
Password-derived key	Derived from a user secret using a password KDF.	Use a suitable KDF and salt; never treat a raw password as a key.
Session key	Protects data during a session or connection.	Limit lifetime and scope; establish it using a secure protocol.

3.3 Security objectives

- Confidentiality: unauthorised users cannot obtain key material or use it to decrypt protected information.
- Integrity and authenticity: changes to encrypted key records or metadata are detected.
- Availability: authorised services can retrieve valid keys when needed, including during planned recovery.
- Accountability: key access and administrative changes are traceable through protected audit events.
- Separation of duties: one compromised account should not automatically provide access to all data and all protecting keys.

4. Problem Identification: Common Flaws and Root Causes

Observed flaw	Root cause	Likely consequence
Keys saved as plain text	Convenience-oriented file or database storage.	Anyone with read access can use the secret.
Key and encrypted data stored together	No separation of key custody.	A single file-system compromise exposes both.
Hard-coded keys or passwords	Secrets committed to source control or copied into examples.	Leaks through repositories, logs, builds, or backups.
Weak or homemade encryption	Outdated algorithms, incorrect modes, missing authentication, or custom crypto.	Confidentiality or integrity may fail.
Predictable key generation	Timestamps, ordinary random functions, or fixed seeds.	An attacker may predict or reproduce the key.
Missing authorisation checks	Authentication confused with permission to use a key.	Any logged-in user may retrieve sensitive keys.
No rotation or revocation	Lifecycle is not modelled or documented.	Compromised keys remain usable indefinitely.
Secrets included in logs/errors	Debug output records values instead of metadata.	Secrets spread into monitoring and support systems.
Unsafe backup/recovery	Backups lack encryption, access controls, or restoration tests.	Keys may leak or become permanently unavailable.

5. Root-Cause Analysis and Debugging Strategy

The debugging process should first establish what fails and under which conditions. A key that cannot be retrieved may indicate a wrong storage path, incorrect encoding, permission denial, key-version mismatch, damaged ciphertext, or a missing protecting key. A key that can be retrieved by an unauthorised user is a more serious access-control defect. These issues should not be treated as interchangeable.

5.1 Step-by-step debugging procedure

1. Reproduce the issue with a disposable test key and non-sensitive test data. Do not debug with production secrets.
2. Map the key path: generation → storage → retrieval → use → rotation → revocation → destruction.

3. Inspect the storage format and confirm that only ciphertext—not plaintext key bytes—is persisted.
4. Check whether the key-encryption key is stored separately and is available only to the intended service identity.
5. Review cryptographic API use: key sizes, encoding, random-number source, authentication/tag verification, and exception handling.
6. Review file, database, service-account, and API permissions. Verify that users cannot bypass application-level checks by reading files directly.
7. Inspect logs and error messages for key bytes, passwords, tokens, or decrypted content. Replace sensitive output with key IDs and event metadata.
8. Create explicit invariants: unauthorised callers must be denied; modified ciphertext must fail authentication; revoked keys must not be used for new operations.
9. Implement one correction at a time and run automated tests after each change.
10. Document the root cause, corrective change, test evidence, residual risk, and operational owner.

5.2 Diagnostic questions

- Does the same key ID consistently map to the same active key version?
- Does retrieval fail because the ciphertext is corrupted, the master key is missing, or permission is denied?
- Can a user read the encrypted key file directly, and if so, can they also obtain the master key?
- Are key values accidentally exposed through debug statements, exception traces, crash dumps, or configuration files?
- Can a revoked key still encrypt new data or authenticate new requests?
- Is there a tested recovery path if the key store or master-key service becomes unavailable?

5.3 Security requirements to verify

Requirement	Verification evidence
Keys are not persisted in plaintext	Storage inspection finds ciphertext only; no secret value appears in logs.
Protecting key is separated	Encrypted store and master-key custody use separate permissions and systems.
Authenticated encryption is used	Tampering with a ciphertext record causes retrieval to fail.
Access is least-privilege	Unauthorised identity receives a denial; authorised service succeeds.
Lifecycle states are enforced	Retired/revoked keys cannot be used for prohibited operations.
Recovery is possible but controlled	Restoration drill succeeds with dual approval and audit trail.

6. Secure Design and Corrective Measures

6.1 Use established cryptographic libraries

Do not invent a cipher, combine cryptographic primitives manually, or rely on obfuscation. Use a maintained library with well-defined APIs. For data that must be encrypted, authenticated encryption is generally preferred because it provides confidentiality and detects tampering. The example later in this report uses Fernet from Python's `cryptography` package, which provides authenticated symmetric encryption for a byte string. The application must still protect the Fernet key itself.

6.2 Separate the encrypted store from the master key

Encrypting a key record is useful only if the key that decrypts it is better protected. The master key should be supplied through a managed secret service, KMS, HSM, or operating-system credential store and made available only to the service identity that needs it. It should not be saved next to the encrypted key file, committed to source control, embedded in a container image, or printed at startup. Environment variables can be a practical development mechanism, but production deployments should use a dedicated secret-delivery mechanism with access policy, auditing, and rotation.

6.3 Apply least privilege and defence in depth

- Give the application only the permissions it needs, and separate read, create, rotate, revoke, and administer privileges where practical.
- Restrict storage permissions at the operating-system or database level, not only in the web interface.
- Require strong authentication for administrators and use multi-factor authentication for sensitive key-management actions.
- Use service identities and short-lived credentials rather than shared administrator accounts.
- Record who performed an operation, when, which key ID/version was affected, and whether it succeeded—but never record the key value.
- Use approval or dual control for high-impact operations such as master-key rotation, bulk export, or destruction.

7. Secure Key Lifecycle Management

A secure key lifecycle defines allowed states, responsible roles, and controls at every stage. A key should never be treated as a permanent string that is created once and forgotten.

Lifecycle stage	Required controls	Typical failure to prevent
1. Generation	Use a cryptographically secure random generator or approved key-generation function; choose algorithm-appropriate parameters.	Predictable or repeated keys.
2. Registration	Assign a unique key ID, algorithm, purpose, owner, creation time, and version.	Ambiguous records and accidental key reuse.
3. Distribution	Use authenticated, encrypted channels or managed key-wrapping; verify recipient identity.	Interception, substitution, or delivery to the wrong party.
4. Storage	Encrypt key material where export is necessary; separate KEK/master key; restrict permissions.	Plaintext exposure or one-compromise access to all keys.
5. Activation and use	Check identity, role, purpose, status, and permitted operation on every sensitive request.	Unauthorised use or use outside intended purpose.
6. Rotation	Create a new version, transition safely, re-encrypt/rewrap as required, and track dependencies.	Long-lived exposure and data becoming unreadable.
7. Deactivation	Stop new use while allowing only approved legacy operations, if needed.	Old key continues to encrypt new data.
8. Revocation	Immediately block use when compromise or policy violation is suspected; propagate status.	Compromised key remains trusted.
9. Backup and recovery	Encrypt backups, separate permissions, document recovery, test restoration.	Permanent loss or backup disclosure.
10. Destruction	Destroy or render unrecoverable all copies according to retention policy; record completion.	Residual copies remain exploitable.

7.1 Rotation without data loss

Rotation must be planned so that data remains decryptable while new encryption uses the new key. A common pattern is to assign each key a stable ID and version. New writes use the active version. Reads select the version that encrypted the record. During migration, old keys may be retained in a restricted decrypt-only state until all dependent data has been re-encrypted or its retention period ends. A key should not be deleted merely because a new version exists.

11. Generate a new key version using a cryptographically secure source.
12. Register it as pending, validate its custody and permissions, and then mark it active.
13. Direct new encryption operations to the active version.
14. Re-encrypt or rewrap existing material according to the design and risk assessment.
15. Verify that all required records can be read with the expected version.
16. Restrict the old version to approved legacy decryption, then revoke and destroy it when no longer required.

7.2 Distribution and revocation

Key distribution should authenticate both the sender and the recipient and protect key material in transit. Public keys may be distributed openly, but their authenticity must be validated through certificates, fingerprints, or a trusted directory. Private and symmetric keys must not be sent in plain email, chat, source-code repositories, or unprotected files. Revocation procedures should define who can initiate revocation, how status reaches dependent services, how cached copies are invalidated, and what emergency actions occur if a master key is suspected to be compromised.

7.3 Backup and recovery

A backup is not secure merely because the primary key store is secure. Backup copies need encryption, separate access control, retention limits, integrity checking, and monitored restoration. Recovery should be tested using a documented procedure and should not depend on one person's memory or one unavailable device. For highly sensitive master keys, split knowledge or dual-control mechanisms may be appropriate. Recovery keys should be protected at least as strongly as the data they unlock.

8. Python Implementation: Encrypted Key Storage

The following educational example demonstrates encrypted storage and retrieval of a test secret using Fernet from the `cryptography` package. It deliberately obtains the protecting key from an environment variable rather than generating a new key on every run or saving the master key beside the encrypted record. In production, a KMS/HSM or operating-system secret manager is preferable to a manually managed environment variable.

8.1 Installation and setup

```
python -m pip install cryptography

# Generate a Fernet key once in a controlled setup process:
python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"

# Set the generated value in your development environment as KEY_STORE_MASTER_KEY.
# Do not commit the value to Git or paste it into source code.
```

For local development, configure `KEY_STORE_MASTER_KEY` in the terminal or IDE's secret configuration. Do not put the real value in a public `.env` file or upload it with the project. If a `.env` file is used locally, exclude it from version control and restrict access.

8.2 Corrected example

```
import json
import os
import tempfile
from pathlib import Path
from cryptography.fernet import Fernet, InvalidToken
```

```
STORE_PATH = Path("encrypted_key_store.json")



class KeyStoreError(Exception):
    """Base exception for safe key-store errors."""



def get_cipher():
    """Load the protecting key from controlled secret configuration."""
    value = os.environ.get("KEY_STORE_MASTER_KEY")
    if not value:
        raise KeyStoreError("Master key is not configured.")
    try:
        return Fernet(value.encode("utf-8"))
    except (ValueError, TypeError) as exc:
        raise KeyStoreError("Master-key configuration is invalid.") from exc



def save_secret(key_id: str, secret: bytes) -> None:
    """Encrypt a test secret before persisting it."""
    if not key_id or not isinstance(secret, bytes) or not secret:
        raise ValueError("A key ID and non-empty byte secret are required.")

    cipher = get_cipher()
    encrypted = cipher.encrypt(secret).decode("ascii")
    record = {
        "key_id": key_id,
        "version": 1,
        "status": "active",
        "ciphertext": encrypted
    }

    # Atomic replacement reduces the chance of a partially written file.
    STORE_PATH.parent.mkdir(parents=True, exist_ok=True)
    fd, temp_name = tempfile.mkstemp(
        prefix="keystore_", suffix=".tmp", dir=str(STORE_PATH.parent)
    )
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as file:
            json.dump(record, file)
            file.flush()
            os.fsync(file.fileno())
        os.replace(temp_name, STORE_PATH)
    finally:
        if os.path.exists(temp_name):
            os.unlink(temp_name)



def retrieve_secret(expected_key_id: str) -> bytes:
    """Retrieve only the requested active record and authenticate it."""
```

```
    try:
        record = json.loads(STORE_PATH.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError) as exc:
        raise KeyStoreError("Key record is unavailable or invalid.") from exc

    if record.get("key_id") != expected_key_id:
        raise KeyStoreError("Requested key ID does not match the record.")
    if record.get("status") != "active":
        raise KeyStoreError("Key is not active for this operation.")

    try:
        return get_cipher().decrypt(record["ciphertext"].encode("ascii"))
    except (InvalidToken, KeyError, ValueError) as exc:
        # Do not reveal key bytes, ciphertext, or internal crypto details.
        raise KeyStoreError("Key record failed authentication or is invalid.") from exc


if __name__ == "__main__":
    # Demonstration only: use a disposable test secret, never a production key.
    save_secret("demo-key-001", b"sample-secret-for-testing")
    recovered = retrieve_secret("demo-key-001")
    print("Key stored and retrieved successfully:", recovered == b"sample-secret-for-testing")
```

8.3 What the correction achieves

- The secret is encrypted before it is written to the file.
- The master key is not written into the same record or hard-coded in the script.
- Fernet authenticates the encrypted data; modified ciphertext should fail decryption.
- A key ID and status field support basic identification and lifecycle checks.
- Atomic file replacement reduces the risk of leaving a partially written record.
- Errors are converted into safe messages that do not print secret material.

Important limitation: this sample is intentionally small. A real multi-key store needs a database or managed KMS, robust schema validation, concurrency control, separate metadata integrity protection, authenticated user/service identities, role checks, rate limiting, audit logging, backups, and a formal rotation and recovery workflow. A single JSON record is not a complete production KMS.

9. Access Control, Storage Hardening, and Secure Retrieval

9.1 Authentication versus authorisation

Authentication answers, “Who is making the request?” Authorisation answers, “Is this identity permitted to perform this operation on this key?” A successful login must not automatically grant permission to export every key. Each request should be checked against a policy that considers identity, role, key purpose, key status, requested operation, and possibly environment or risk context.

Role	Typical allowed operations	Operations to restrict
Application service	Use an assigned key for a narrow cryptographic operation.	Export master key, change policies, destroy keys.
Key administrator	Create versions, schedule rotation, review metadata.	Read application plaintext or unrestricted private key material by default.
Security auditor	Read audit events and policy metadata.	Export or use secret key material.
Recovery custodian	Participate in approved recovery workflow.	Unilateral unrestricted access where dual control is required.
Ordinary user	Use application functions authorised for their account.	Direct key-store access or administrative key operations.

9.2 Storage hardening

- Store key material in a dedicated secret manager, KMS, or HSM whenever available.
- If a local file is used for a demonstration, restrict the containing directory and file to the service account; on Windows, configure NTFS ACLs rather than assuming POSIX permission calls apply.
- Encrypt backups and test restoration. Do not create unprotected temporary copies.
- Keep encrypted records, master keys, logs, and backups under separate access policies where practical.
- Use TLS with certificate validation for remote key-service communication.
- Disable unnecessary key export. Prefer cryptographic operations performed inside the KMS/HSM when supported.
- Keep the cryptographic library and runtime patched; pin and review dependencies.
- Avoid printing secrets in logs, telemetry, debugging output, exception messages, or test reports.

9.3 Safe error handling and audit logs

Errors should provide enough information for operators to diagnose a failure without disclosing secrets. For example, an audit event can contain a timestamp, actor identity, key ID, version, operation, result, and correlation ID. It should not contain the key bytes, password, bearer token, plaintext data, or a full decrypted object. Logs should be access-controlled, integrity-protected, retained according to policy, and monitored for repeated denials, unexpected exports, unusual rotation activity, and attempts to use revoked keys.

10. Optimisation Without Weakening Security

The task requests optimisation, but cryptographic performance should not be improved by weakening key length, disabling authentication, reusing nonces incorrectly, skipping certificate checks, or storing plaintext. Optimisation should target avoidable I/O, repeated configuration loading, unnecessary serialization, excessive remote calls, and inefficient lifecycle operations.

Optimisation	Benefit	Security condition
Reuse a configured KMS client	Reduces repeated connection setup and latency.	Refresh credentials safely; do not cache secrets indefinitely.
Use key IDs and versions	Avoids scanning records and reduces lookup cost.	Metadata must be validated and access-controlled.
Cache non-secret metadata	Reduces database round trips.	Invalidate promptly on revocation or policy changes.
Use envelope encryption	Limits exposure of bulk-data keys and simplifies rewrapping.	Protect KEKs separately and define version mapping.
Batch administrative operations	Reduces API overhead during controlled rotation.	Use bounded batches, audit each result, and support rollback.
Atomic writes and transactions	Prevents partial records and inconsistent states.	Preserve durability and validate before commit.
Measure before tuning	Identifies actual bottlenecks.	Never bypass authentication or integrity checks to improve benchmark results.

11. Testing and Validation Plan

Testing must demonstrate both functional correctness and security properties. A successful round trip alone is insufficient: the system must also reject unauthorised requests, detect tampering, handle missing configuration safely, and respect lifecycle state.

Test ID	Test case	Expected result
T01	Store a disposable test secret and retrieve it with the correct ID.	Recovered bytes match the test secret.
T02	Open the persisted file and inspect the record.	Plaintext secret is not present; ciphertext is present.
T03	Retrieve using a different key ID.	Request is rejected.
T04	Change one character in ciphertext.	Authenticated decryption fails; no plaintext is returned.
T05	Remove the master-key configuration.	Operation fails safely with a configuration error.
T06	Use an invalid master-key value.	Configuration is rejected; no secret is printed.
T07	Set key status to revoked in a controlled test fixture.	Retrieval/use is rejected by lifecycle policy.
T08	Attempt access using an identity without permission.	Access denied and security event recorded.
T09	Interrupt a write or simulate a full disk.	Existing record remains intact where atomic replacement is supported.
T10	Rotate from version 1 to version 2.	New writes use v2; required legacy reads remain controlled.
T11	Restore an encrypted backup in a test environment.	Authorised recovery succeeds and is audited.
T12	Search logs and repository history for secret patterns.	No real key or password is exposed.

11.1 Example test harness

```
# Example pytest-style tests (adapt to the application's module name)
import json
import pytest
from cryptography.fernet import Fernet


def test_round_trip(monkeypatch, tmp_path):
    # In a real test, inject the store path rather than using a global path.
    monkeypatch.setenv("KEY_STORE_MASTER_KEY", Fernet.generate_key().decode())
    # save_secret("test-1", b"temporary-test-secret")
    # assert retrieve_secret("test-1") == b"temporary-test-secret"
    assert True  # Replace with the assertions above in the project test suite.


def test_missing_master_key(monkeypatch):
    monkeypatch.delenv("KEY_STORE_MASTER_KEY", raising=False)
    # assert the public API raises KeyStoreError without printing secrets.
    assert True  # Replace with an assertion against the application's API.
```

The harness above is a test-plan template, not a complete automated test suite: the sample implementation should be refactored to inject the storage path and cipher provider so that tests can use temporary directories and disposable keys. Tests must never contain production secrets.

11.2 Performance measurements

Measure storage and retrieval latency separately, and report median and high-percentile values rather than only one run. Also measure throughput under concurrent requests, failure rate, memory use, KMS API calls, and time required to rotate a defined number of keys. Compare the original and corrected versions using the same machine, data size, library versions, and workload. The expected result is not necessarily that encryption itself is dramatically faster; the main improvement is secure and predictable operation with reduced unnecessary I/O and fewer repeated setup calls.

Metric	How to measure	Interpretation
Store latency	Time from validated request to durable encrypted record.	Includes encryption and storage overhead.
Retrieval latency	Time from authorised request to verified key availability.	Includes lookup, access checks, and decryption.
Failure rate	Failed valid operations / all valid operations.	Should remain low without masking security denials.
Tamper detection	Number of modified records correctly rejected.	All deliberately altered test records should be rejected.
Rotation duration	Time to create, activate, migrate, and verify a batch.	Must preserve readability and auditability.
Unauthorised success count	Count of denied-test identities that gain access.	Target is zero.

12. Analysis: How Poor Key Management Compromises a System

Poor key management can nullify the protections offered by encryption and digital signatures. The consequences are not restricted to the key-store component; any data, service, identity, or protocol that trusts the exposed key may be affected.

12.1 Confidentiality impact

If a data-encryption key is disclosed, an attacker may decrypt stored or intercepted ciphertext that the key protects. If the key has been reused across many records or environments, the scope of disclosure becomes larger. If an encrypted backup and its master key are stored together, theft of that backup may be enough to recover the data. Key separation, narrow scope, rotation, and access monitoring reduce the likelihood and impact of this event.

12.2 Integrity and authenticity impact

An exposed signing private key may allow an attacker to create fraudulent signatures. A compromised authentication or message-authentication key may allow forged messages to be accepted. If encrypted key records are not authenticated, an attacker may alter or substitute data without the system reliably detecting the change. Authenticated encryption and trusted key identifiers help detect tampering, but they do not compensate for an attacker who has obtained the legitimate key.

12.3 Availability and recovery impact

Loss or corruption of a key can make encrypted information permanently unreadable. Accidental deletion, premature destruction, or a failed rotation can be as damaging as disclosure. An attacker who can delete or revoke critical keys may cause an outage. Secure backup, tested restoration, staged rotation, and tightly controlled administrative permissions are therefore availability controls as well as security controls.

12.4 Trust, compliance, and financial impact

A key compromise can require incident response, credential revocation, data re-encryption, customer notification, forensic investigation, and service interruption. It can also undermine trust in signed software, certificates, identity systems, or encrypted communications. The exact legal or regulatory obligations depend on the organisation, jurisdiction, data type, and applicable standards; this report does not assume a specific compliance regime.

12.5 Risk summary

Threat	Likelihood drivers	Impact	Primary mitigation
Plaintext key theft	Broad file access; backups; source-control leaks.	Critical confidentiality loss.	Managed secret storage, encryption, least privilege.
	Key stored beside ciphertext;	Potential compromise of many	KMS/HSM, separation of duties,

Master-key compromise	overprivileged identity.	protected keys.	access monitoring.
Key substitution/tampering	No authentication or metadata validation.	Wrong key used; integrity/authenticity failure.	Authenticated encryption, trusted IDs, integrity checks.
Stale compromised key	No rotation, revocation, or status propagation.	Continuing unauthorised decryption or signing.	Lifecycle state enforcement and incident procedures.
Permanent data loss	No backup or untested recovery.	Loss of access to protected data.	Encrypted backup, controlled recovery drills.
Insider misuse	Shared accounts and unrestricted export.	Unauthorised access or exfiltration.	Individual identities, least privilege, audit and dual control.

13. Recommended Implementation Workflow

17. Inventory all keys, algorithms, owners, purposes, storage locations, dependencies, and current permissions.
18. Classify key material by sensitivity and define who may create, use, export, rotate, revoke, and destroy each class.
19. Move master-key custody to a KMS/HSM or approved secret manager; remove hard-coded values and rotate any secret that may already have leaked.
20. Encrypt stored key records with authenticated encryption where export is unavoidable; separate the protecting key from the records.
21. Enforce authentication and operation-specific authorisation at the service boundary and storage layer.
22. Implement key IDs, version metadata, lifecycle states, audit events, and a documented rotation workflow.
23. Add tests for tampering, denial of access, revoked keys, missing configuration, backup recovery, and rotation.
24. Benchmark normal operations and rotation; optimise only measured bottlenecks without removing security checks.
25. Perform a security review, dependency review, recovery drill, and final audit before deployment.

14. Results and Discussion

The proposed corrected design addresses the assessment's central failure—unsafe storage and retrieval—by ensuring that key material is encrypted before persistence, that the protecting key is not stored in the same record, that retrieval checks the requested identifier and lifecycle status, and that authentication failures are handled without returning plaintext. It also establishes operational requirements that a small code patch alone cannot satisfy.

Assessment criterion	Evidence supplied in this answer	Expected outcome
Problem identification (20%)	Root-cause table and diagnostic questions.	Clear mapping between defects, causes, and consequences.
Debugging (25%)	Repeatable debugging procedure and corrected Python example.	Storage/retrieval faults are isolated and corrected systematically.
Optimisation (20%)	Performance strategies and measurement plan.	Reduced avoidable overhead without weakening controls.
Analysis (20%)	Confidentiality, integrity, availability, and trust analysis.	Security impact and mitigations are logically justified.
Code quality (15%)	Structured functions, validation, exception handling, comments, and tests.	Readable code with clear limitations and testable behaviour.

14.1 Expected versus measured results

No execution environment, original faulty Python program, production key store, or benchmark dataset was included in the question. Therefore, this report does not claim measured speedups or present invented experimental numbers. The expected outcomes above are design and test expectations. For a complete practical submission, run the implementation and test suite in the intended environment, record actual timings and test results, and replace the expected-outcome statements with observed evidence.

14.2 Limitations

- The Fernet demonstration uses a single encrypted record and is not a complete multi-user key-management service.
- The example does not implement a production identity provider, database transactions, distributed revocation, HSM integration, or approval workflow.
- Environment variables are shown for demonstration convenience; production master-key custody should use a managed facility.
- The sample's status field illustrates lifecycle checks, but in production the status metadata itself must be protected against unauthorised modification.
- Exact retention periods, rotation intervals, algorithm choices, and compliance obligations must be set by a documented risk assessment and applicable organisational policy.

15. Conclusion

A key management system is a foundational security component because it controls the secrets on which encryption, authentication, key agreement, and digital signatures depend. Plaintext storage, hard-coded keys, weak encryption, insecure distribution, missing access controls, and incomplete lifecycle handling can compromise an entire cryptographic system even when the underlying algorithms are strong.

The proposed solution combines systematic debugging with secure design: use a maintained cryptographic library; encrypt persisted key material with authenticated encryption; keep the master key separate; enforce least privilege; track key IDs, versions, and lifecycle states; protect backups; rotate and revoke keys in a controlled manner; and test tampering, unauthorised access, recovery, and performance. Operational improvements should reduce unnecessary work without disabling authentication, integrity checks, or access control.

The most important conclusion is that key security is a lifecycle responsibility. Secure storage is necessary but not sufficient. A dependable implementation also requires trusted key generation, authenticated distribution, controlled use, monitored rotation, emergency revocation, tested recovery, and verified destruction. These controls reduce the chance that one software defect, compromised account, or lost file can expose the entire system.

16. References

National Institute of Standards and Technology. (2020). Recommendation for key management: Part 1 – General (NIST Special Publication 800-57 Part 1, Revision 5). https://doi.org/10.6028/NIST.SP.800-57pt1r5

National Institute of Standards and Technology. (2019). Recommendation for key management: Part 2 – Best practices for key management organizations (NIST Special Publication 800-57 Part 2, Revision 1). https://doi.org/10.6028/NIST.SP.800-57pt2r1

National Institute of Standards and Technology. (2020). Recommendation for password-based key derivation (NIST Special Publication 800-132). https://doi.org/10.6028/NIST.SP.800-132

National Institute of Standards and Technology. (2007). Recommendation for block cipher modes of operation: The CMAC mode for authentication (NIST Special Publication 800-38B). https://doi.org/10.6028/NIST.SP.800-38B

OWASP Foundation. (n.d.). Secrets Management Cheat Sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html

Python Cryptographic Authority. (n.d.). Fernet (symmetric encryption). Cryptography documentation. https://cryptography.io/en/latest/fernet/

Python Software Foundation. (n.d.). secrets — Generate secure random numbers for managing secrets. Python documentation. https://docs.python.org/3/library/secrets.html

National Institute of Standards and Technology. (2020). Security and privacy controls for information systems and organizations (NIST Special Publication 800-53, Revision 5). https://doi.org/10.6028/NIST.SP.800-53r5

Appendix A. Submission Checklist

- Replace the cover-page placeholders with the student's details.
- If required by the instructor, add screenshots of successful storage/retrieval and failed tampering tests.
- Include the actual faulty code and a before/after comparison if it was provided separately.
- Run the code with disposable test secrets and document the Python and cryptography package versions.
- Add measured benchmark results rather than estimated or invented performance claims.
- Ensure no real cryptographic keys, passwords, tokens, or private data appear in the submitted document or screenshots.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation